

Lab 3

Task 0

```
#include "gtest/gtest.h"

int add(int a, int b) { return a + b; }

class TestAdd : public ::testing::Test
{
    void SetUp() override
    {
        // Runs before each test
    }

    void TearDown() override
    {
        // Runs after each test
    }
};

// Don't worry about the syntax here, the TEST_F macro is very complicated.
// Just know that this is how you create a test case.
TEST_F(TestAdd, AddTest)
{
    // This should pass, 2 + 4 = 6
    EXPECT_EQ(add(2, 4), 6);
}

TEST_F(TestAdd, AddTest2)
{
    // Create a test case here. Maybe fail this to see what happens?
    EXPECT_EQ(add(3, 3), 9); //failing testcase
}
```

```

int main(int argc, char **argv)
{
    // Standard Google Test main function
    testing::InitGoogleTest(&argc, argv);
    auto res = RUN_ALL_TESTS();
    return res;
}

```

The above code will return:

```

[=====] Running 2 tests from 1 test suite.
[-----] Global test environment set-up.
[-----] 2 tests from TestAdd
[ RUN    ] TestAdd.AddTest
[   OK   ] TestAdd.AddTest (0 ms)
[ RUN    ] TestAdd.AddTest2
main.cpp:31: Failure
Expected equality of these values:
  add(3, 3)
    Which is: 6
  9
[ FAILED ] TestAdd.AddTest2 (0 ms)
[-----] 2 tests from TestAdd (0 ms total)

[-----] Global test environment tear-down
[=====] 2 tests from 1 test suite ran. (0 ms total)
[ PASSED ] 1 test.
[ FAILED ] 1 test, listed below:
[ FAILED ] TestAdd.AddTest2

1 FAILED TEST

```

However, after correcting the test case, we would get the message saying that we have passed all test cases:

```

[=====] Running 2 tests from 1 test suite.
[-----] Global test environment set-up.
[-----] 2 tests from TestAdd
[ RUN    ] TestAdd.AddTest
[   OK   ] TestAdd.AddTest (0 ms)
[ RUN    ] TestAdd.AddTest2
[   OK   ] TestAdd.AddTest2 (0 ms)
[-----] 2 tests from TestAdd (0 ms total)

[-----] Global test environment tear-down
[=====] 2 tests from 1 test suite ran. (0 ms total)
[ PASSED ] 2 tests.

```

Task 1

4-bit LFSR and Pseudo Random Binary Sequence

```

module lfsr(
    input logic    clk,
    input logic    rst,
    input logic    en,
    output logic [3:0] data_out
);

logic [3:0] sreg;

always_ff @ (posedge clk, posedge rst)
    if (rst)
        //setting the initial variables of sreg because rst is on at the start of testbench
        sreg <= 4'b1;
    else
        sreg <= {sreg[2:0], sreg[3]^ sreg[2]};
assign data_out = sreg;

```

```
endmodule
```

Running the code, we get successful verification

```
[=====] Running 3 tests from 1 test suite.  
[-----] Global test environment set-up.  
[-----] 3 tests from TestDut  
[ RUN    ] TestDut.InitialStateTest  
[    OK  ] TestDut.InitialStateTest (0 ms)  
[ RUN    ] TestDut.SequenceTestMini  
[    OK  ] TestDut.SequenceTestMini (0 ms)  
[ RUN    ] TestDut.SequenceTest  
[    OK  ] TestDut.SequenceTest (0 ms)  
[-----] 3 tests from TestDut (0 ms total)  
  
[-----] Global test environment tear-down  
[=====] 3 tests from 1 test suite ran. (0 ms total)  
[ PASSED ] 3 tests.
```

Extending this for the 7 bit lfsr circuit. Note that the 7th order primitive is $1 + X^3 + X^7$

```
module lfsr_7 (  
    input logic  clk,  
    input logic  rst,  
    input logic  en,  
    output logic [6:0] data_out  
);  
  
logic [6:0] sreg;  
  
always_ff @ (posedge clk, posedge rst)
```

```

    if (rst)
        sreg <= 7'b1;
    else
        sreg <= {sreg[6:0], sreg[2] ^ sreg[6]};
    assign data_out = sreg;

endmodule

```

Task 2

Formula 1 Light Sequence

```

module f1_fsm (
    input  logic    rst,
    input  logic    en,
    input  logic    clk,
    output logic [7:0] data_out
);

logic [7:0] output_reg;

typedef enum {S0, S1, S2, S3, S4, S5, S6, S7, S8} my_state;
my_state current_state, next_state;

always_ff @ (posedge clk, posedge rst)
    if (rst)    current_state <= S0;
    else       current_state <= next_state;

always_comb
    case (current_state)
        S0:      if (en)    next_state = S1;
                  else     next_state = current_state;
        S1:      if (en)    next_state = S2;
                  else     next_state = current_state;

```

```

S2:    if (en)    next_state = S3;
        else      next_state = current_state;
S3:    if (en)    next_state = S4;
        else      next_state = current_state;
S4:    if (en)    next_state = S5;
        else      next_state = current_state;
S5:    if (en)    next_state = S6;
        else      next_state = current_state;
S6:    if (en)    next_state = S7;
        else      next_state = current_state;
S7:    if (en)    next_state = S8;
        else      next_state = current_state;
S8:    if (en)    next_state = S0;
        else      next_state = current_state;
default: next_state = S0;
endcase

```

always_comb

```

case(current_state)
S0:    output_reg = 8'b0;
S1:    output_reg = 8'b1;
S2:    output_reg = 8'b11;
S3:    output_reg = 8'b111;
S4:    output_reg = 8'b1111;
S5:    output_reg = 8'b11111;
S6:    output_reg = 8'b111111;
S7:    output_reg = 8'b1111111;
S8:    output_reg = 8'b11111111;
default: output_reg = 8'b0;
endcase

```

assign data_out = output_reg;

endmodule

Running the code on the verification file

```
[=====] Running 2 tests from 1 test suite.  
[-----] Global test environment set-up.  
[-----] 2 tests from TestDut  
[ RUN    ] TestDut.InitialStateTest  
[   OK   ] TestDut.InitialStateTest (0 ms)  
[ RUN    ] TestDut.FSMTest  
[   OK   ] TestDut.FSMTest (0 ms)  
[-----] 2 tests from TestDut (0 ms total)  
  
[-----] Global test environment tear-down  
[=====] 2 tests from 1 test suite ran. (0 ms total)  
[ PASSED ] 2 tests.
```

test bench file, note that we have to set vflag to one shot mode so that we cycle on click

```
#include "verilated.h"  
#include "verilated_vcd_c.h"  
#include "Vf1_fsm.h"  
#define MAX_SIM_CYC 1000000  
  
#include "vbuddy.cpp" //include vbuddy code  
  
int main(int argc, char **argv, char **env) {  
    int simcyc;    // simulation clock count  
    int tick;      // each clk cycle has two ticks for two edges  
  
    Verilated::commandArgs(argc, argv);  
    // init top verilog instance  
    Vf1_fsm* top = new Vf1_fsm;  
    // init trace dump  
    Verilated::traceEverOn(true);
```

```

VerilatedVcdC* tfp = new VerilatedVcdC;
top→trace (tfp, 99);
tfp→open ("f1_fsm.vcd");

// init Vbuddy
if (vbdOpen()!=1) return(-1);
vbdHeader("L3T2: F1 Light Sequence");
vbdSetMode(1);

// initialize simulation inputs
top→clk = 1;
top→rst = 1;
top→en = 0;

// run simulation for MAX_SIM_CYC clock cycles
for (simcyc=0; simcyc<MAX_SIM_CYC; simcyc++) {
    // dump variables into VCD file and toggle clock
    for (tick=0; tick<2; tick++) {
        tfp→dump (2*simcyc+tick);
        top→clk = !top→clk;
        top→eval ();
    }

    top→en = vbdFlag();
    top→rst = (simcyc < 2);

    // plot data_out from module
    vbdBar(top→data_out & 0xFF);

    // either simulation finished, or 'q' is pressed
    if ((Verilated::gotFinish()) || (vbdGetkey()=='q'))
        exit(0);          // ... exit if finish OR 'q' pressed
}

vbdClose();    // ++++
tfp→close();

```



```

delete top;
exit(0);
}

```

Task 3

Exploring the clktick.sv and the delay.sv modules

Optimal N value is 24 (done by testing with a metronome)

```

//optimal N value is 24

module clktick #(
    parameter WIDTH = 16
)(
    // interface signals
    input logic      clk,    // clock
    input logic      rst,    // reset
    input logic      en,     // enable signal
    input logic [WIDTH-1:0] N, // clock divided by N+1
    output logic      tick   // tick output
);

    logic [WIDTH-1:0] count;

    always_ff @ (posedge clk)
        if (rst) begin // common practice to have being/end for all if
            tick <= 1'b0; // output a 0
            count <= N; // set count back to N
        end
        else if (en) begin
            if (count == 0) begin
                tick <= 1'b1; // set output to 1
                count <= N; // reset count
            end
        end
        else begin

```

```

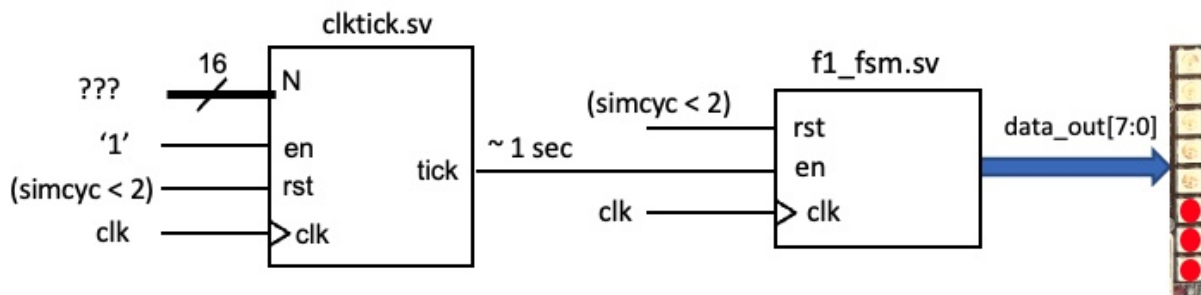
        tick <= 1'b0;          // set output back to 0
        count <= count - 1'b1; // decrement count by 1
    end
end
endmodule

```

The **vbdValue()** shown on bottom left of the TFT display is the value for N which gives a tick period of 1 second! (Why?)

- 60 bpm means there are 60 beats per minute which would mean 1 bps (beat per second) and therefore signify a period of 1 second.

Challenge



Top level file

```

module f1_1s_cycle (
    input logic    en,
    input logic    rst,
    input logic    clk,
    input logic [15:0] n_val,
    output logic [7:0] data_out
);

```

```

    logic tick; //internal connection between tick output and en input

```

```

    clktick one_second_tick(
        .clk(clk),

```

```

        .rst(rst),
        .en(en),
        .N(n_val),
        .tick(tick)
    );

    f1_fsm f1_light_sequence(
        .rst(rst),
        .en(tick),
        .clk(clk),
        .data_out(data_out)
    );

endmodule

```

Note that we needed to change both modules so that it is synchronous reset. The test bench is as follows, we have set 'en' to be read from the flag (we turned off one shot mode).

```

#include "Vf1_1s_cycle.h"
#include "verilated.h"
#include "verilated_vcd_c.h"

#include "../vbuddy.cpp" // include vbuddy code
#define MAX_SIM_CYC 100000

int main(int argc, char **argv, char **env)
{
    int simcyc;    // simulation clock count
    int tick;      // each clk cycle has two ticks for two edges
    int lights = 0; // state to toggle LED lights

    Verilated::commandArgs(argc, argv);
    // init top verilog instance
    Vf1_1s_cycle *top = new Vf1_1s_cycle;
    // init trace dump

```

```

Verilated::traceEverOn(true);
VerilatedVcdC *tfp = new VerilatedVcdC;
top->trace(tfp, 99);
tfp->open("f1_1s_cycle.vcd");

// init Vbuddy
if (vbdOpen() != 1)
    return (-1);
vbdHeader("L3T2: f1 1sec cycle");
vbdSetMode(0); // Flag mode set to continuous (not one-shot)

// initialize simulation inputs
top->clk = 1;
top->rst = 0;
top->en = 0;
top->n_val = 24;

// run simulation for MAX_SIM_CYC clock cycles
for (simcyc = 0; simcyc < MAX_SIM_CYC; simcyc++)
{
    // dump variables into VCD file and toggle clock
    for (tick = 0; tick < 2; tick++)
    {
        tfp->dump(2 * simcyc + tick);
        top->clk = !top->clk;
        top->eval();
    }

    top->rst = (simcyc < 2); // assert reset for 1st cycle
    top->en = vbdFlag(); //en is based on vbd flag
    vbdBar(top->data_out & 0xFF); //light setting

    if (Verilated::gotFinish() || vbdGetkey() == 'q')
        exit(0);
}

```

```
vbdClose(); // ++++  
tfp→close();  
exit(0);  
}
```